Programación II

Profesorado de Informática

CERP del Suroeste

Sección "Cómo llegamos hasta aquí" de UML Gota a Gota.

Docente

Prof. Domingo Pérez González

Estudiante

Camila Fleitas Pérez

Tareas: Las presentes tareas deben realizarse en orden -o no- prestando atención a la relación entre las lecturas realizadas.

a. Leer el Prefacio de UML Gota a Gota.

b. Leer la sección “Cómo llegamos hasta aquí” de UML Gota a Gota y crea un documento que expanda lo escrito hasta nuestros días.

c. Aprende UML utilizando Visual Paradigm https://www.visual-paradigm.com/guide/ y crea un resumen conciso de los conceptos más relevantes trabajados en clase. Explora los tutoriales y guías disponibles en el sitio web, enfocándote en los temas que se relacionan con los conceptos discutidos en clase. A medida que avances, toma notas y organiza un resumen estructurado de los conceptos clave de UML. Finalmente, comparte y discute tus hallazgos con tus compañeros de clase para mejorar la comprensión de UML y su aplicación en proyectos futuros.

Rb): En el texto “Cómo llegamos hasta aquí”, Fowler y Scott explican cómo fue evolucionando el desarrollo orientado a objetos hasta llegar a UML. A finales del siglo XX existían diferentes métodos y formas de representar los sistemas, lo que generaba cierta competencia entre ellos, conocida como la “guerra de métodos”.

Entre los principales referentes se encontraban Grady Booch, James Rumbaugh, con su metodología OMT, e Ivar Jacobson, con OOSE. Cada uno proponía su propia forma de trabajar y representar los sistemas. Con el tiempo, estos enfoques comenzaron a unificarse y, a mediados de los años 90, los tres trabajaron juntos en Rational Software para desarrollar un lenguaje común. Así surgió UML (Unified Modeling Language), que posteriormente fue adoptado y estandarizado por el Object Management Group (OMG) en 1997.

A partir de ese momento, UML continuó evolucionando junto con la ingeniería de software. Una de las etapas importantes fue la aparición de UML 2.0, que amplió las posibilidades del lenguaje y permitió representar sistemas cada vez más complejos.

Con el crecimiento de las metodologías ágiles, como Scrum y Kanban, también cambió la forma de utilizar UML. En lugar de realizar una gran cantidad de diagramas como documentación formal, comenzaron a utilizarse principalmente para representar ideas, explicar partes de un sistema y facilitar la comunicación entre los integrantes de un equipo.

Actualmente, UML sigue siendo utilizado, aunque su uso puede variar según el proyecto y la metodología de trabajo. También existen herramientas que permiten crear diagramas a partir de texto, como PlantUML y Mermaid, lo que facilita mantener los diagramas junto con el código y actualizarlos durante el desarrollo del proyecto.

Rc): A partir de la documentación y las guías de Visual Paradigm, se puede entender UML como un lenguaje que permite representar visualmente diferentes aspectos de un sistema. Se utiliza para analizar, diseñar, comunicar y documentar las características de un sistema, pero no es un lenguaje de programación ni una metodología de desarrollo.

Los diagramas UML se pueden dividir principalmente en dos grupos: diagramas estructurales y diagramas de comportamiento.

Los diagramas estructurales permiten representar cómo está formado un sistema. Dentro de este grupo se encuentra el diagrama de clases, que es uno de los más utilizados. En él se pueden representar las clases, sus atributos, métodos y las relaciones que existen entre ellas.

Por otro lado, los diagramas de comportamiento permiten representar cómo funciona el sistema y cómo se comportan sus elementos. Entre ellos se encuentran los diagramas de casos de uso, que muestran las acciones que pueden realizar los usuarios o actores dentro del sistema; los diagramas de secuencia, que representan el intercambio de mensajes entre los objetos en un determinado orden; los diagramas de actividades, que permiten mostrar los pasos de un proceso; y los diagramas de estados, que representan los diferentes estados por los que puede pasar un objeto.

También es importante conocer las relaciones que se pueden establecer entre los elementos de UML. Por ejemplo, la asociación permite representar una relación entre clases. La inclusión y la extensión se utilizan principalmente en los casos de uso. La generalización representa una relación de herencia entre una clase general y otras más específicas. Por último, la agregación y la composición representan relaciones entre un todo y sus partes, aunque se diferencian por el grado de dependencia que existe entre ellas.